



TRABAJO FINAL

Cliente de Correo Electrónico



MARÍA LARA ZOBAJI – MAURICIO RAMIREZ
GRUPO 29 – COMISIÓN 2

Trabajo Final – Cliente de Correo Electrónico

Cátedra: Estructuras de Datos en Python UNaB –
Comisión 2

Integrantes: María Lara Zobaji y Mauricio Ramírez

*Este documento presenta el análisis formal de la **complejidad computacional (Big O)** de las operaciones críticas implementadas en el Cliente de Correo Electrónico. Se justifica el uso de cada estructura de datos central (Árbol General, Cola de Prioridad y Grafo) en función de la eficiencia y el cumplimiento de los requisitos del proyecto.*

2. Justificación de Estructuras de Datos

2.1 Árbol General (Gestión de Carpetas)

Se eligió un modelo de **Árbol General** para representar la jerarquía de carpetas de cada usuario.

- **Implementación:** La clase **Carpeta** utiliza un diccionario (`_subcarpetas: Dict[str, Carpeta]`) para almacenar sus nodos hijos, permitiendo que cada carpeta tenga múltiples subcarpetas.
 - **Complejidad de Operaciones:**
 - **Crear Carpeta:** $O(1)$. La inserción es instantánea si se tiene la referencia al nodo padre.
 - **Buscar Mensaje Recursivo:** $O(N)$. En el peor caso (N = número total de mensajes/carpetas), el sistema debe visitar cada elemento para asegurar que se encuentre el mensaje en cualquier parte de la jerarquía.
-

2.2 Cola de Prioridad (Mensajes Urgentes)

Se utiliza una Cola de Prioridad para gestionar los mensajes marcados como "urgentes".

- **Implementación:** La clase **ServidorCorreo** utiliza la librería **heapq** de Python, que implementa un *min-heap* binario, para la lista de mensajes pendientes (`_mensajes_en_espera`). La prioridad se define en el método `__lt__` de la clase **Mensaje**.
- **Decisión de Diseño:** Esta estructura garantiza que los mensajes urgentes se procesen primero, ya que el elemento de mayor prioridad siempre está disponible en la raíz del *heap*.

- **Eficiencia (n = elementos en la cola):**
 - **Encolar Mensaje (Inserción):** $O(\log n)$. El costo es logarítmico, manteniendo el servidor rápido sin ralentizarse por muchos correos.
 - **Procesar/Extraer Mensaje (Desencolar):** $O(\log n)$ La extracción del mensaje más urgente es igualmente eficiente.
-

2.3 Grafo de Servidores (Red Distribuida)

El Grafo modela la infraestructura de la red de servidores de correo.

- **Implementación:** La clase **RedServidores** utiliza un modelo de lista de adyacencia. Los servidores son **vértices** (nodos) y las conexiones entre ellos son **aristas**.
 - **Algoritmos de Enrutamiento:** Se implementaron dos algoritmos para encontrar la ruta de envío.
 - **BFS (Búsqueda en Amplitud):** Usado para encontrar la **ruta más corta** (menor cantidad de saltos).
 - **DFS (Búsqueda en Profundidad):** Usado para explorar caminos, priorizando la profundidad sobre la amplitud.
 - **Complejidad:**
 - **Encontrar Ruta (BFS/DFS):** $O(V + E)$. La complejidad es lineal respecto al número de **Vértices** (V) y **Aristas** (E) en la red, lo que garantiza que se encuentre el camino de manera eficiente.
-

3. Complejidad de Funcionalidades Avanzadas

3.1 Aplicación de Filtros Configurables

La nueva lógica de filtros permite al usuario definir reglas que mueven los mensajes entrantes a carpetas específicas, utilizando un diccionario para almacenar las reglas²¹.

- **Implementación:** La clase Usuario almacena los filtros en un diccionario mapeado por ID. Al recibir un mensaje, el sistema itera sobre la lista de reglas configuradas por el usuario.
- **Complejidad:** $O(m)$. La complejidad es lineal respecto al número m de reglas que el usuario haya configurado. Esto es muy eficiente, ya que la aplicación de filtros no depende del número total de mensajes en el sistema, sino únicamente del número de reglas.